Mayank Heda
Roll Number: 35 T12
AI&DS ASD&D Lab

Experiment 1

LO2: Use git effectively for version control source code managements and collaborative managements

Introduction to Git and GitHub

1. Global Username Configuration

PowerShell
git config --global user.name "mayank-heda"

- **Purpose:** Sets the global Git username configuration. This ensures that every future code snapshot (commit) created on this machine is officially signed and tracked under the author profile `mayank-heda` .

2. Initial Email Configuration

PowerShell
git config --global user.email "your-email@example.com"

- **Purpose:** Sets an initial placeholder email address for the global configuration.

3. Username Reinforcement

PowerShell
git config --global user.name "mayank-heda"

- **Purpose:** Re-executed to verify and lock in the global author username profile configuration.

4. Corrected Email Configuration

PowerShell
git config --global user.email "mayank@gmail.com"

- **Purpose:** Overwrites the placeholder email configuration with the correct personal email address (`mayank@gmail.com`) ensuring lab submissions and cloud platforms correctly identify the committer.

5. Listing Active Configurations

PowerShell
git config --list

- **Purpose:** Displays the full active configuration profile of the local Git environment. It was used here to verify that `user.name=mayank-heda` and `user.email=mayank@gmail.com` were successfully saved in the system.

6. Local Repository Initialization

PowerShell
git init

- **Purpose:** Initialized a brand new, empty local Git repository in the active directory path (`C:/Users/Lab 203/.git/`). This creates the hidden tracker system files required to begin version control on the local machine.

7. Erroneous Repository Clone Attempt

PowerShell
git clone https://github.com/mayank-heda/experiment_1.git

- **Purpose:** Attempted to download a remote project structure. This command failed with a `fatal: repository not found` error due to a syntax typo where an underscore (`experiment_1`) was used instead of a hyphen.

8. Successful Remote Repository Cloning

PowerShell
git clone https://github.com/mayank-heda/experiment-1.git

- **Purpose:** Downloads the target online repository directly onto the local workstation machine. The console generated a warning stating `You appear to have cloned an empty repository`, confirming the operation was successful and a clean workflow directory named `experiment-1` was successfully linked to GitHub.

9. Repository Status Verification & Directory Scope Check

PowerShell
git status

- **Purpose:** Checks the current state of the active repository timeline. Because this was inadvertently run inside the system's **User Home folder** (`C:\Users\Lab 203`) instead of inside the project subfolder, Git scanned the entire user directory.
- **Analysis of Console Output:**

    - **Permission Warnings:** The terminal generated multiple `Permission denied` warnings because Git attempted to scan protected Windows system directories (such as `Application Data` and `Local Settings`).
    - **State Context:** The output `On branch master` and `No commits yet` shows that the root User Home directory had an unconfigured Git tracking context initialization.
    - **Untracked Files (Red Output):** Git listed the entire contents of the computer's user folder—including application settings, system configurations, internal caches, and the `experiment-1/` project folder—identifying them all as loose, untracked modifications. This step highlighted that the directory location needed to be updated using a change-directory command before continuing.

10. Empty Script File Initialization

PowerShell
New-Item main.py -ItemType File

- **Purpose:** Dynamically creates a brand-new, empty file named `main.py` directly inside the targeted `experiment-1` project directory (`C:\Users\Lab 203\experiment-1`). The console output displays the file metadata, confirming a file length of 0 bytes.

11. Project Content Staging

PowerShell
git add .

- **Purpose:** Stages all newly created or modified assets within the project workspace folder. This marks the newly created empty `main.py` file as ready to be packaged into the subsequent repository history snapshot.

12. Root Commit Execution

PowerShell
git commit -m "Add initial files for experiment-1"

- **Purpose:** Saves the staged baseline changes permanently to the local Git timeline history ledger. The terminal output notes the creation of a master root-commit tagged with the unique identifier `8cc79bb`.

13. Failed Remote Syncing (Authentication Conflict)

PowerShell
git push -u origin main

- **Purpose:** Attempted to upload the local commit timeline up to the remote GitHub cloud server. The execution failed with a `fatal: 403 Forbidden` error because the system attempted to authenticate using a cached credential set (`astha9500`) which lacks permission to write to the target `mayank-heda` project repository.

14. Successful Remote Integration & Re-Authentication

PowerShell
git push -u origin main

- **Purpose:** Re-executed the cloud sync operation after removing the invalid cached system tokens. The console prompted browser-based user authorization, established the proper credentials, securely uploaded the data packages to GitHub, and set up the local `main` branch to track the remote `origin/main` server track.

15. Redundant File Creation Attempt

PowerShell
New-Item main.py -ItemType File

- **Purpose:** An accidental attempt to re-create the base script file. The operating system blocked execution and threw a red `WriteError: IOException` warning declaring that the target file `main.py` already exists in that path.

16. Redundant Staging and Commit Checks

PowerShell
git add .
git commit -m "Add initial files for experiment-1"

- **Purpose:** Attempted to compile a secondary snapshot out of caution. Git evaluated the directory tracking tree, recognized no new content changes had occurred since the last operation, and returned `nothing to commit, working tree clean`.

17. Final Sync Verification

PowerShell
git push -u origin main

- **Purpose:** Checks the synchronization status between the local workstation and the GitHub cloud server. The output `Everything up-to-date` confirms that both storage contexts match completely.

18. Check Unstaged File Differences

PowerShell

git diff

- **Purpose:** Analyzes the directory for modified file text. Since no text modifications had been written yet, the command returned blank, confirming no local drift existed.

19. Local Branch Enumeration

PowerShell
git branch

- **Purpose:** Lists available development branches. The output `* main` verified that only the default development timeline existed and was actively selected.

20. Isolated Feature Branch Creation

PowerShell
git branch feature-optimization

- **Purpose:** Creates a secondary, parallel development branch named `feature-optimization`. This creates an isolated sandbox environment to safely script changes without altering the stable `main` timeline.

21. Branch Environment Selection

PowerShell
git switch feature-optimization

- **Purpose:** Switches the active working environment from `main` to the newly established `feature-optimization` timeline branch.

22. Redundant Duplicate Branch Creation Conflict

PowerShell
git checkout -b feature-optimization

- **Purpose:** An attempt to create and switch branches simultaneously. Git blocked the action and generated a `fatal` error message stating that a branch named `feature-optimization` already exists.

23. Preemptive Branch Merging Test

PowerShell
git merge feature-optimization

- **Purpose:** Attempts to pull developments from the feature branch into the active workspace tracking. The terminal returned `Already up to date` because no new commits had been written to either timeline.

24. Blocked Branch Deletion Attempt

PowerShell
git branch -d feature-optimization

- **Purpose:** Attempted to delete the feature branch. Git blocked the process with an error stating `cannot delete branch... used by worktree`, preventing the user from deleting a branch they are actively standing inside.

25. Return to Primary Timeline Context

PowerShell
git switch main

- **Purpose:** Switches the workspace target environment back to the primary `main` branch timeline so subsequent maintenance actions can be performed.

26. Successful Feature Branch Deletion Clean-up

PowerShell
git branch -d feature-optimization

- **Purpose:** Safely deletes the now-vacated `feature-optimization` branch. The output logs that the branch was cleanly dropped at baseline point `8cc79bb`.

27. Redundant Remote Mapping Conflict

PowerShell
git remote add origin https://github.com/mayank-heda/experiment-1.git

- **Purpose:** Attempted to establish a new link to the online repository. The console threw an `error: remote origin already exists` because the initial cloning step had already automated this path connection mapping.

28. Remote Sync Verification

PowerShell
git push -u origin main

- **Purpose:** Uploads local timeline states up to GitHub to confirm consistency, returning `Everything up-to-date`.

29. Remote Content Integration Sync

PowerShell
git pull

- **Purpose:** Checks the cloud server for new changes and integrates them. The response `Already up to date` verifies no remote changes had been submitted by external sources.

30. Remote Connection URL Inspection

PowerShell
git remote -v

- **Purpose:** Lists the explicit fetch and push network URLs mapped to the `origin` identifier, confirming secure tracking links lead straight to `mayank-heda/experiment-1.git` .

31. Simplified History Tracking Output

PowerShell
git log --oneline

- **Purpose:** Generates a compact list of history records. The console verified that local `main`, cloud `origin/main`, and head pointers all sat uniformly at root baseline commit `8cc79bb`.

32. Staging Revocation Pathspec Failure

PowerShell
git restore --staged code.py

- **Purpose:** Attempted to pull a file out of the staging context area. The operation failed with a `pathspec... did not match any file(s)` error because no file named `code.py` existed anywhere within the repository folder.

33. Successful Script Staging Revocation

PowerShell
git restore --staged main.py

- **Purpose:** Correctly target-unstages `main.py`, removing it from the index queue without affecting its code content.

34. Uncommitted Change Discard Reversion

PowerShell
git restore main.py

- **Purpose:** Wipes out any unstaged text modifications inside `main.py`, forcing it to instantly match its baseline saved properties exactly as recorded in the history ledger.

35. Script Creation with Inline Values

PowerShell
New-Item main.py -ItemType File -Value 'print("Hello, World!")'

- **Purpose:** Creates a python script file (`main.py`) directly inside the `experiment-1` project directory and populates it instantly with a simple hello world print instruction. The terminal metadata shows a length of 22 bytes, confirming the text data was successfully written to the disk.

36. Workspace Change Staging

PowerShell
git add .

- **Purpose:** Instructs Git to scan the local folder for modifications and stage the newly updated `main.py` script file, prepping it to be recorded in the local historical ledger.

37. Script Architecture Commit Execution

PowerShell
git commit -m "Add simple hello world script"

- **Purpose:** Packages the staged script changes and saves them permanently into a local history snapshot. The terminal logs that the commit was assigned the unique tracking hash ID `21904fd`, recording 1 file modification and 1 line insertion.

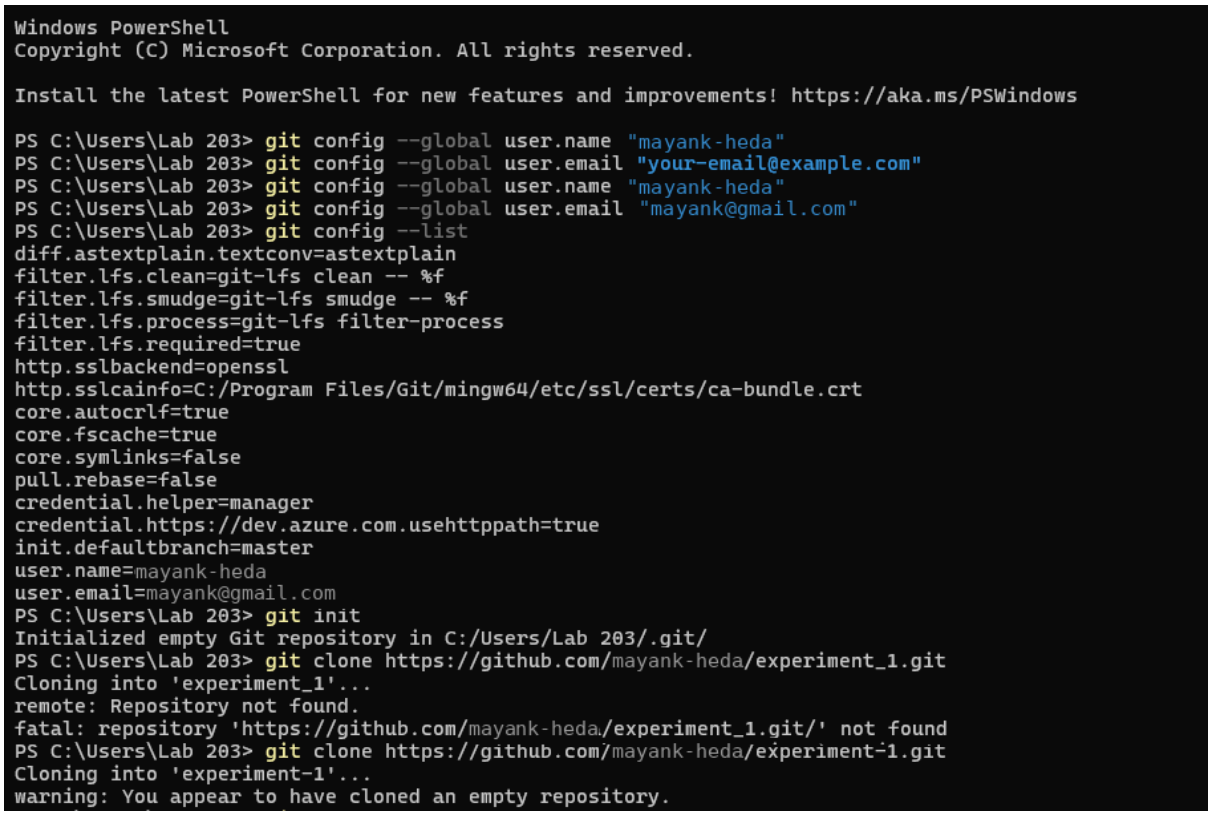
38. Final Production Remote Syncing

PowerShell
git push origin main

- **Purpose:** Uploads the local script history timeline up to the remote GitHub cloud server repository. The terminal tracks the data packing progression and confirms a successful push update, syncing the main timeline drift from previous position `6402268` to the newly established tracking peak `21904fd`.

Conclusion:

This experiment successfully demonstrates the foundational workflow of Git version control by configuring user credentials, initializing local environments, and managing baseline code repositories. Through hands-on execution, core lifecycle operations—including file staging, history tracking logs, branching sandboxes, and timeline rollbacks—were systematically practiced and validated. Ultimately, the local project tracking structure was seamlessly connected and synchronized with the remote GitHub cloud interface, ensuring a reliable system for distributed engineering collaboration and project backup.



```
PS C:\Users\Lab 203> git status
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .AltairRapidMiner/
        .anaconda/
        .bash_history
        .conda/
        .condarc
        .continuum/
        .dbus-keyrings/
        .docker/
        .gitconfig
        .gradle/
        .idlerc/
        .ipynb_checkpoints/
        .ipython/
        .jupyter/
        .lesshst
        .local/
        .m2/
        .matplotlib/
        .vscode/
        3D Objects/
        AppData/
        Atlassian/
        Contacts/
        Desktop/
        Documents/
        Downloads/
        Favorites/
        IntelGraphicsProfiles/
        Links/
        Music/
        NTUSER.DAT
        NTUSER.DAT{2ad838bc-efea-11ee-a54d-000d3a94eaa1}.TM.blf
        NTUSER.DAT{2ad838bc-efea-11ee-a54d-000d3a94eaa1}.TMContainer00000000000000000001.regtrans-ms
        NTUSER.DAT{2ad838bc-efea-11ee-a54d-000d3a94eaa1}.TMContainer00000000000000000002.regtrans-ms
        OneDrive/
        Pictures/
        Saved Games/
        Searches/
        TensorFlow/
        Untitled.ipynb
        Untitled1.ipynb
        Videos/
        anaconda3/
        anaconda_projects/
        app.py
        experiment-1/
        git-dvcs/
        git-dvcs69/
        git-t13-61/
        javac
        main.py
        myproject/
        ntuser.dat.LOG1
        ntuser.dat.LOG2
```

```
PS C:\Users\Lab 203\experiment-1> New-Item main.py -ItemType File


    Directory: C:\Users\Lab 203\experiment-1


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        14-07-2026     10:26              0 main.py


PS C:\Users\Lab 203\experiment-1> git add .
PS C:\Users\Lab 203\experiment-1> git commit -m "Add initial files for experiment-1"
[main (root-commit) 8cc79bb] Add initial files for experiment-1
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 main.py
PS C:\Users\Lab 203\experiment-1> git push -u origin main
remote: Permission to mayank-heda/experiment-1.git denied to astha9500.
fatal: unable to access 'https://github.com/mayank-heda/experiment-1.git/': The requested URL returned error: 403
PS C:\Users\Lab 203\experiment-1> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 231 bytes | 231.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/mayank-heda/experiment-1.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Lab 203\experiment-1> New-Item main.py -ItemType File
New-Item : The file 'C:\Users\Lab 203\experiment-1\main.py' already exists.
At line:1 char:1
+ New-Item main.py -ItemType File
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : WriteError: (C:\Users\Lab 203\experiment-1\main.py:String) [New-Item], IOException
    + FullyQualifiedErrorId : NewItemIOError,Microsoft.PowerShell.Commands.NewItemCommand

PS C:\Users\Lab 203\experiment-1> git add .
PS C:\Users\Lab 203\experiment-1> git commit -m "Add initial files for experiment-1"
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS C:\Users\Lab 203\experiment-1> git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
```

```
PS C:\Users\Lab 203\experiment-1> git diff
PS C:\Users\Lab 203\experiment-1> git branch
* main
PS C:\Users\Lab 203\experiment-1> git branch feature-optimization
PS C:\Users\Lab 203\experiment-1> git switch feature-optimization
Switched to branch 'feature-optimization'
PS C:\Users\Lab 203\experiment-1> git checkout -b feature-optimization
fatal: a branch named 'feature-optimization' already exists
PS C:\Users\Lab 203\experiment-1> git merge feature-optimization
Already up to date.
PS C:\Users\Lab 203\experiment-1> git branch -d feature-optimization
error: cannot delete branch 'feature-optimization' used by worktree at 'C:/Users/Lab 203/experiment-1'
PS C:\Users\Lab 203\experiment-1> git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\Lab 203\experiment-1> git branch -d feature-optimization
Deleted branch feature-optimization (was 8cc79bb).
PS C:\Users\Lab 203\experiment-1> git remote add origin https://github.com/mayank-heda/experiment-1.git
error: remote origin already exists.
PS C:\Users\Lab 203\experiment-1> git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
PS C:\Users\Lab 203\experiment-1> git pull
Already up to date.
PS C:\Users\Lab 203\experiment-1> git remote -v
origin  https://github.com/mayank-heda/experiment-1.git (fetch)
origin  https://github.com/mayank-heda/experiment-1.git (push)
PS C:\Users\Lab 203\experiment-1> git log --oneline
8cc79bb (HEAD -> main, origin/main, origin/HEAD) Add initial files for experiment-1
PS C:\Users\Lab 203\experiment-1> git restore --staged code.py
error: pathspec 'code.py' did not match any file(s) known to git
PS C:\Users\Lab 203\experiment-1> git restore --staged main.py
PS C:\Users\Lab 203\experiment-1> git restore main.py
```

```
PS C:\Users\Lab 203\experiment-1> New-Item main.py -ItemType File -Value 'print("Hello, World!")'


    Directory: C:\Users\Lab 203\experiment-1


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        14-07-2026     11:04             22 main.py


PS C:\Users\Lab 203\experiment-1> git add .
PS C:\Users\Lab 203\experiment-1> git commit -m "Add simple hello world script"
[main 21904fd] Add simple hello world script
 1 file changed, 1 insertion(+)
 create mode 100644 main.py
PS C:\Users\Lab 203\experiment-1> git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Writing objects: 100% (3/3), 277 bytes | 277.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/mayank-heda/experiment-1.git
   6402268..21904fd  main -> main
PS C:\Users\Lab 203\experiment-1>
```